

**ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ**

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И  
ТЕХНОЛОГИИ“**



**СИСТЕМА ЗА АНАЛИЗ НА ТРАНСПОРТНАТА  
СИГУРНОСТ И ЗА УПРАВЛЕНИЕ НА  
ДОПУСКАНЕТО НА ВРЪЗКИ ПРИ УСЛОВИЯ  
НА ЛАВИНЕН ТРАФИК**

**КУРСОВ ПРОЕКТ**

**по дисциплина „Мрежова и информационна сигурност“**

**Разработил:**

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

**Преподавател:**

Проф. д-р инж. Румен Трифонов

София, **[TODO: година на предаване]**

# СЪДЪРЖАНИЕ

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>УВОД</b> . . . . .                                        | <b>1</b>  |
| <b>I. ПРЕДМЕТНА ОБЛАСТ И ИЗБОР НА РЕШЕНИЯ</b> . . . . .      | <b>2</b>  |
| 1.1. Стандарти, заплахи и протоколи . . . . .                | 2         |
| 1.2. Приети решения и цената им . . . . .                    | 2         |
| <b>II. ПРОЕКТИРАНЕ И ПРОГРАМНА РЕАЛИЗАЦИЯ</b> . . . . .      | <b>4</b>  |
| 2.1. Разчитане и граница на видимостта . . . . .             | 4         |
| 2.2. Проверка на сертификатната верига . . . . .             | 4         |
| 2.3. Допускане, стенд и тестове . . . . .                    | 5         |
| 2.4. Модел на заплахата и граници . . . . .                  | 6         |
| <b>III. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА</b> . . . . .   | <b>7</b>  |
| <b>IV. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ</b> . . . . .      | <b>8</b>  |
| 4.1. Цена на едно решение за допускане . . . . .             | 8         |
| 4.2. Поведение при лавинен трафик . . . . .                  | 9         |
| 4.3. Отчитане на очакванията и какво не е измерено . . . . . | 10        |
| <b>ЗАКЛЮЧЕНИЕ</b> . . . . .                                  | <b>11</b> |
| <b>ЛИТЕРАТУРА</b> . . . . .                                  | <b>12</b> |
| <b>ПРИЛОЖЕНИЕ</b> . . . . .                                  | <b>13</b> |

## УВОД

Сигурността на един мрежов обмен се крепи на два механизма, които потребителят не вижда: протоколът за защита на транспортния слой и инфраструктурата на публичните ключове. И двата се реализират от малък брой библиотеки и точно затова свойствата им рядко се проверяват самостоятелно. Приема се, че щом връзката е установена, тя е защитена, и че щом библиотеката не е върнала грешка, веригата от сертификати е валидна.

Проектът изхожда от обратната постановка. Договорените параметри на една връзка са наблюдаема величина. Валидността на една верига е резултат от алгоритъм, записан в [1], който може да бъде реализиран самостоятелно. Устойчивостта срещу лавинен трафик е свойство, което се измерва, а не се декларира.

**Целта** е работеща и измерима система, наречена Sentinel, която чете ръкостискането по TLS 1.3, проверява сертификатни вериги по алгоритъма от стандарта и управлява допускането на нови връзки при претоварване, като цената на всеки механизъм е измерена, а не предположена. **Задачите** са пет: преглед на приложимите стандарти и на заплахите; обосновка на избора между възможните решения; проектиране на слоеста архитектура; реализация на C++20, в която непроведените проверки се отчитат изрично като непроведени; и измерване в собствена изолирана среда. Към тях стои изискване, което се оказва не по-малко определящо: сглобяване на чужда машина без нито една задължителна външна зависимост.

**Обхватът** е защитен. Системата анализира, проверява и допуска. Тя не съдържа средства за проникване, а опитите с лавинен трафик се провеждат срещу собствен стенд, чийто слушател се свързва само с адреса на локалния интерфейс.

**Ограничение, което важи за целия документ.** Системата не е подлагана на атака, на фъзинг или на външен одит. Всяко твърдение по-нататък се отнася до това, което кодът проверява, отчита и измерва, а не до доказана устойчивост срещу нападател.

# **I. ПРЕДМЕТНА ОБЛАСТ И ИЗБОР НА РЕШЕНИЯ**

## **1.1. Стандарти, заплахи и протоколи**

Рамката за управление на информационната сигурност идва от серията ISO/IEC 27000. Пряко приложима тук е ISO/IEC 27033-1 [2], която разделя мрежата на зони и описва как се избират мерки за границата между тях. Форматът на сертификата за публичен ключ е дефиниран в препоръката ITU-T X.509 [3], а профилът му за интернет в [1]. Указанията на NIST за настройка на TLS [4] и за политиката на защитните стени [5] са полезни с това, че формулират проверими твърдения: изискване от вида „да не се договаря версия под TLS 1.2“ анализаторът на ръкостискането може да провери автоматично.

От класификацията на заплахите в доклада на ENISA [6] значение тук имат три. Отказът на услуга, включително разпределеният му вариант. Злоупотребата с инфраструктурата на публичните ключове, при която проверяващата страна приема сертификат, който не би трябвало да приеме. И манипулацията на разрешаването на имена: ако името е разрешено до грешен адрес, проверката на веригата остава единственият механизъм, който може да открие подмяната, защото удостоверяване на произхода на отговора липсва по устройство [7]. Класификацията на OWASP [8] е ползвана само като контролен списък какво остава извън обхвата: проектът работи под приложния слой.

Протоколните спецификации са три. [9] определя докъде стига пасивният анализ, защото TLS 1.3 шифрира по-голямата част от ръкостискането. [10] обяснява защо лавината от заявки за връзка е евтина за източника и скъпа за получателя, и описва контрамерките заедно със страничните им ефекти; тези странични ефекти са величината, която проектът измерва. [11] показва, че същата ключова обмяна се пренася и в друг транспорт, тоест механизмите не са свързани с TCP.

## **1.2. Приети решения и цената им**

Едно решение предопределя останалите: от какво има право да зависи сборката. Обичайният подход е библиотеките да идват от управител на пакети, при което реализацията е по-кратка, но сборката изисква подготвена машина и мрежа. Приет е обратният, защото хранилището е публично и трябва да може да бъде сглобено от непознат без подготовка. Цената е платена на три места: отпадат библиотеката за прихващане на пакети, външната рамка за тестове и проверката на подписа.

**Източник на данните.** Анализът отвътре в приложението описва само собствените връзки и е обвързан с версията на криптографската библиотека. Пасивното прихващане на пакети е общо, но внася задължителна зависимост. Приетото решение е слой за разчитане да приема просто изглед към байтове: общността и възпроизводимостта на входа се запазват, зависимостта отпада. Загубено е удобството да се чете формат на записан трафик направо, което е отделна задача за разчитане на формат, а не свойство на анализа.

**Обхват на собствената реализация.** Криптографските примитиви са зона, в която собствената реализация е грешка: изборът на алгоритми, устойчивостта срещу атаки по странични канали и правилността на изчисленията са резултат от години рецензии. Замисълът предвиждаше примитивите да дойдат от утвърдена библиотека, но това влезе в противоречие с изискването за сглобяване без подготовка. От трите изхода, а именно задължителна криптографска зависимост, собствена аритметика с големи числа или непроведена проверка, е приет третият: проверката на подписа не се извършва и това се обявява при всяка верига. Първите два или отнемат свойството, заради което хранилището е използваемо, или заменят рецензирана реализация с нерецензирана. Съществено е, че обявяването е част от изхода: мълчаливото пропускане на проверка прави отчета по-опасен от липсата на отчет. По същата причина отмяната е сведена до местен списък от серийни номера, тъй като запитване по [12] изисква изходяща връзка, каквато системата няма.

**Стратегия за допускане.** Стойностите без съхранение на състояние [10] премахват нуждата от състояние за незавършена връзка, но изискват допълнителен обмен. Ограничаването по източник е евтино, но е безполезно срещу разпределен източник и вредно, когато честният и лавинният трафик делят един адрес. Изчислителната задача прехвърля цената върху клиента, но внася закъснение при първо свързване. Отчитането на таблицата с връзките е по-скоро измерителен механизъм, но без него останалите три не могат да бъдат оценени. Приети са и четирите, включвани поотделно, защото въпросът не е кой механизъм е най-добър, а колко струва всеки, което изисква сравнение при еднакви условия.

## II. ПРОЕКТИРАНЕ И ПРОГРАМНА РЕАЛИЗАЦИЯ

Системата е разделена на четири слоя, всеки проверим самостоятелно: четене на байтове, разчитане на протокола, проверка на веригата и допускане на връзки. Обемът по модули е в Приложението.

### 2.1. Разчитане и граница на видимостта

Входът е `std::span<const std::uint8_t>`, тоест невладеещ изглед към чужд буфер, което прави измерването възпроизводимо. Всяко поле минава през четец, който проверява дължината предварително; при недостиг четецът се маркира като отказал и причината за първия отказ се запазва, без изключения: изкривеното съобщение е нормален вход за такъв код. Дължината на записа се проверява срещу границата от [9], раздел 5.1, преди копирането на какъвто и да е байт, иначе изпращачът решава колко памет заделя получателят. Съдържанието на всички записи от вид `handshake` се слепва в един буфер, преди да бъде нарязано на съобщения, защото едно съобщение може да се разпростре върху няколко записа; разчитач, който приравнява запис на съобщение, сгрешава точно интересния случай.

В явен вид са само `ClientHello` и `ServerHello`: предложени и договорени версия, шифрова съвкупност, група за ключова обмяна, схеми за подпис, име на сървъра и `ALPN`. Скрити остават `Certificate`, `CertificateVerify` и `Finished`, а с тях сертификатните вериги на двете страни. Анализаторът докладва наблюдаваното и изброява какво не е видял. Версията се чете от `supported_versions`, не от `legacy_version`: по [9] последното остава със стойност за TLS 1.2 при всяка връзка по TLS 1.3, за да преминава през междинни устройства. Разчитачът на DER отхвърля неопределена дължина, неминимално кодиране и високия номер на етикета: разчитач, който приема няколко кодирания на една стойност, е начинът, по който две реализации стигат до различно мнение какво пише в сертификата.

### 2.2. Проверка на сертификатната верига

Първият етап изгражда пътя до доверен корен чрез обхождане в дълбочина по кандидат-издателите, като разглежда всички продължения, а не само първото намерено: първият намерен път до корен често не е този, който минава проверките. Вторият прилага проверките от [1] (Табл. 1), при което резултатът има три състояния вместо две: `PASS`,

FAIL и SKIP. Третото позволява да се каже коя проверка не е проведена и защо, вместо тя да бъде преброена като успешна.

**Таблица 1. Проверки върху пътя и състоянието им при изрядна верига от три сертификата**

| Проверка                  | Раздел   | Какво отхвърля                      | Състояние |
|---------------------------|----------|-------------------------------------|-----------|
| Свързване по име          | 6.1      | прекъсване между издател и субект   | PASS      |
| Срок на валидност         | 4.1.2.5  | изтекъл или още невлязъл в сила     | PASS      |
| Основни ограничения       | 4.2.1.9  | издател, който не е удостоверявател | PASS      |
| Дължина на пътя           | 4.2.1.9  | повече звена от позволеното         | PASS      |
| Предназначение на ключа   | 4.2.1.3  | удостоверявател без keyCertSign     | PASS      |
| Разширено предназначение  | 4.2.1.12 | ключ, негоден за сървър             | PASS      |
| Съответствие на името     | RFC 6125 | име, непокрито от subjectAltName    | PASS      |
| Критични разширения       | 6.1      | неразпознато критично разширение    | PASS      |
| Отмяна                    | 5        | сериен номер в местния списък       | PASS      |
| Ограничения върху имената | 4.2.1.10 | име извън позволеното поддърво      | SKIP      |
| Подпис на издателя        | 6.1      | подправен подпис                    | SKIP      |

Двата реда със SKIP са най-важното в таблицата. Първият е такъв само по този път, тъй като никой издател по него не ограничава имена. Вторият е постоянен: подписът не се проверява, както е обяснено в Раздел 1.2, и всеки отчет носи ред SKIP с причината. Съответствието на името следва RFC 6125, раздел 6.4.3: един заместващ знак, само в най-лявата съставка, никога през точка, и поне две съставки след него.

### 2.3. Допускане, стенд и тестове

Слоят е верига от четири независими механизма, всеки включван отделно. Първа е кофата с жетони по източник, най-евтината проверка. Втора е бисквитката без състояние, siphash24 върху източника, стойността на клиента и груб времеви прозорец: при първи опит сървърът не заделя нищо, а връща стойност, която клиентът трябва да върне и с това да докаже, че получава на адреса, който твърди, че има. Трета е изчислителната задача, при която проверката у сървъра е едно хеширане, а търсенето у клиента е средно  $2^k$  хеширания

при  $k$  бита трудност. Едва четвърта е таблицата с връзките, тоест заделянето на състояние; при запълване се изхвърля най-старата незавършена връзка, защото при лавина тя почти сигурно е част от лавината. Политиката на плавно влошаване вдига цената на първите три с нарастване на запълването.

Слушателят на стенда се свързва единствено с адреса на локалния интерфейс: генераторът на натоварване не може да достигне нищо освен този процес. Списъкът с 82 теста се произвежда от самия изпълним файл, за да не може да се размине с кода, а целият входен материал се генерира от хранилището срещу [9] и [1]; сертификатите носят запълващ низ на мястото на подписа, който никой не чете.

## 2.4. Модел на заплахата и граници

Приема се, че наблюдателят няма частните ключове и не може да дешифрира защитената част от обмена. (Табл. 2) изброява срещу какво системата действа и срещу какво не действа.

Таблица 2. Модел на заплахата: актьор, способност и мярка на системата

| Актьор                                    | Способност                                  | Мярка                                                         |
|-------------------------------------------|---------------------------------------------|---------------------------------------------------------------|
| Пасивен наблюдател                        | чете трафика, няма ключове                  | извън обхвата: системата е на негово място                    |
| Неправомерен издател                      | подписва сертификат за чуждо име            | съответствие и ограничения върху имената, основни ограничения |
| Страна с негодна верига                   | поддържа произволни сертификати             | път до доверен корен, проверка на всяко звено                 |
| Подправен подпис при верни полета         | преподписва съдържанието                    | <b>няма мярка:</b> подписът не се проверява                   |
| Лавина от един адрес                      | евтино открива връзки                       | кофа с жетони по източник, отчитане на таблицата              |
| Разпределена лавина                       | открива връзки от много адреси              | бисквитка без състояние, задача, плавно влошаване             |
| Нападател над машината или приложния слой | изпълнява код, злоупотребява над транспорта | извън модела, вж. [8]                                         |

Мерките в третата колона са проектни решения, проверени от тестовите върху построени случаи, но не срещу действителен нападател: системата не е подлагана на атака, на фъзинг и на външен одит.



### III. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА

Разделът е написан преди измерванията, за да остане погрешното очакване видимо. Клиентът и сървърът работят в един процес върху локалния интерфейс, тоест никакъв трафик не напуска машината: AMD Ryzen 5 3600 с 12 логически процесора, 15,9 GiB памет, Windows 11 Pro N, g++ 15.2.0 (MinGW-w64), Release с -O3 -DNDEBUG. Машината е обща: няма закрепване към ядро и по време на опитите на нея работят други процеси.

Анализаторът се проверява върху обмен, разкъсан между два записа, върху сървър със сигнал за понижаване и върху прекъснато съобщение. Проверката на веригата се провежда върху собствена йерархия от тринадесет случая с по един нарочен дефект, сред които изтекъл сертификат, непокрито име, заместващ знак през точка, издател, който не е удостоверител, надхвърлена дължина на пътя и отменен сериен номер; три от случаите трябва да бъдат приети. Очакваният изход е записан при построяването на случая.

Разчитането и проверката се измерват в затворен цикъл без сокети, след загряване. Цената на едно решение се сема също в затворен цикъл, при който подаваното време расте с една микросекунда на решение: замразен часовник би изпразнил кофата и би превърнал остатъка в измерване на пътя на отказа. Изчислителната задача се измерва от двете страни поотделно, защото смесването им би скрило несиметричността. Поведението при лавинен трафик се сема върху действителни TCP връзки, с две нишки честни клиенти и четири нишки, които отварят връзка и никога не отговарят. Параметрите са: 11 повторения по 600 ms, таблица с 2048 записа, задържане 20 ms след допускане, трудност 12 бита, кофа с вместимост 4096 и попълване 2 000 000 в секунда.

Измерването е валидно при две условия: генераторът да не е тясното място, което се проверява по решенията за секунда, и междуквартилният размах (МКР) да остава под предварително определен праг, 25 на сто при разчитане и проверка, 35 на сто за едно решение и 40 на сто за допуснатите връзки. Ред над своя праг носи надпис, че не бива да бъде докладван като точна стойност. Предварителните очаквания, отчетени в Раздел IV, са пет: бисквитката ще намали заделеното състояние срещу допълнителен обмен (O1); ограничаването по източник няма да раздели честния от лавинния трафик при споделен адрес (O2); задачата ще е силно несиметрична (O3); цената на решението ще е малка спрямо цената на връзката (O4); разчитането ще е от порядъка на микросекунди (O5).

## IV. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

Всяко число идва от едно изпълнение на `sentinel_bench --duration-ms 600 --repeat 11` на машината от Раздел III. Пълният изход, с всички единадесет повторения, е в хранилището като `docs/measurements/bench_run.txt`, тоест всяка обобщена стойност може да бъде проследена до стойностите зад нея. Докладваната стойност е медиана.

**Таблица 3. Пропускателна способност на разчитането и на проверката**

| Операция                              | ns за операция | операции/s | МКР    | размах |
|---------------------------------------|----------------|------------|--------|--------|
| Разчитане на обмен по TLS 1.3, 2448 В | 7 765,4        | 128 776    | 11,5 % | 31,9 % |
| Разчитане на сертификат, 984 В        | 8 575,7        | 116 608    | 8,0 %  | 15,8 % |
| Изграждане и проверка на верига от 3  | 17 330,4       | 57 702     | 6,7 %  | 13,7 % |

И трите реда са под своя праг от 25 на сто. Проверката на верига струва два пъти повече от разчитането на един сертификат, като разликата е в изграждането на пътя и в проверките, част от които изграждат канонично представяне на отличителните имена. Числото е важно със своя порядък: при 57 хиляди проверки в секунда на едно ядро проверката на сертификати не е тясното място при никое натоварване, което една машина може да поеме по ТСП. Присъдите съвпадат с очакваните за всичките тринадесет построени случая, което се проверява от отделен тест при всяка сборка. Това важи само за построените случаи: те не заместват атака срещу системата.

### 4.1. Цена на едно решение за допускане

(Табл. 4) дава цената на едно решение вътре в процеса, без сокет по пътя, тоест собствената цена на механизма, отделена от цената на връзката.

Първият ред надхвърля своя праг от 35 на сто и **не бива да бъде докладван като точна стойност**: при базовата конфигурация решението е увеличаване на брояч и връщане, тоест единици наносекунди, а върху обща машина разсейването на планировчика е от същия порядък. От останалите редове се четат четири величини: отчитането струва около 85 ns, тоест цената на заделеното състояние в хеш-таблицата; ограничаването по източник около 1 ns над него, разлика под разсейването, от която не се вади заключение освен

**Таблица 4. Цена на едно решение за допускане, измерена вътре в процеса**

| Конфигурация                  | ns за решение | МКР    | размах | у клиента |
|-------------------------------|---------------|--------|--------|-----------|
| Без механизъм (база)          | 8,7           | 40,7 % | 50,7 % | няма      |
| Отчитане                      | 93,9          | 8,8 %  | 15,1 % | няма      |
| Отчитане + ограничаване       | 94,9          | 9,4 %  | 18,7 % | няма      |
| Отчитане + бисквитка          | 124,9         | 13,7 % | 31,9 % | няма      |
| Отчитане + бисквитка + задача | 147,8         | 7,3 %  | 49,0 % | 0,05 ms   |
| Всички включени               | 136,0         | 32,2 % | 54,2 % | 0,06 ms   |

това, че цената е малка; бисквитката около 31 ns, тоест две извиквания на SipHash-2-4; и проверката на изчислителната задача около 23 ns, тоест едно хеширане. Търсенето у клиента при 12 бита трудност отнема средно 4279 хеширания и 0,05 ms, тоест отношение малко над 2000. Ако трудността се вдигне с един бит, работата у клиента се удвоява, а цената у сървъра остава същата; точно затова плавно влошаване вдига трудността. Редът „Всички включени“ дава по-малко от реда без ограничаване по източник, но разликата е в рамките на разсейването. Отбелязва се, защото премълчаването на неудобно подреден ред е начинът, по който една таблица става недостоверна.

## 4.2. Поведение при лавинен трафик

**Таблица 5. Цена на механизмите за допускане при лавинен трафик спрямо базата**

| Конфигурация                  | допуснати/s | спрямо базата | p50, us | p95, us | върхова таблица | памет, B |
|-------------------------------|-------------|---------------|---------|---------|-----------------|----------|
| Без механизъм (база)          | 4305        | 100,0 %       | 454,2   | 575,9   | 0               | 0        |
| Отчитане                      | 3614        | 83,9 %        | 524,9   | 762,8   | 287             | 7552     |
| Отчитане + ограничаване       | 3450        | 80,1 %        | 516,3   | 858,3   | 255             | 7912     |
| Отчитане + бисквитка          | 1919        | 44,6 %        | 987,8   | 1390,2  | 48              | 1504     |
| Отчитане + бисквитка + задача | 1671        | 38,8 %        | 1094,2  | 1840,1  | 44              | 1408     |
| Всички включени               | 1791        | 41,6 %        | 1027,1  | 1692,8  | 43              | 1384     |

Съпоставката между двете таблици дава най-съществения извод. Собствената цена на едно решение е от 9 до 148 наносекунди, а цената на една допусната връзка под натоварване е от 454 до 1094 *микросекунди*, тоест около четири порядъка разлика. Намалението на пропускателната способност не идва от изчислението, а от формата на

протокола: базата взема 12 919 решения в секунда, а конфигурацията с бисквитка 11 532. Приемникът е еднакво зает; променя се колко от решенията завършват с връзка.

Бисквитката сваля допуснатите връзки до 44,6 на сто от базата и удвоява закъснението, защото една връзка изисква два обмена вместо един: първият опит получава само предизвикателство и се затваря. Срещу тази цена върховото запълване на таблицата пада от 287 записа на 48, а заделената памет за състояние от 7552 на 1504 байта, при едно и също количество лавинен трафик. Сделката, изразена в измерени числа: половината пропускателна способност срещу шесткратно намаление на състоянието, което лавината може да наложи.

Ограничаването по източник дава 80,1 срещу 83,9 на сто за самото отчитане, тоест разлика в рамките на разсейването. По-същественото наблюдение е качествено: върху локалния интерфейс честните и лавинните клиенти делят един адрес, така че ограничител, който захваща, би задушил и двата вида. Това не е недостатък на стенда, а проявление на известното ограничение на механизма: срещу разпределен източник кофата с жетони не работи, а срещу споделен адрес тя вреди. Добавянето на изчислителната задача сваля допуснатите връзки до 38,8 на сто и вдига медианата с около 106 микросекунди.

#### **4.3. Отчитане на очакванията и какво не е измерено**

И петте очаквания се потвърждават: O1 от падането на върховата таблица от 287 на 48 записа, O2 от 80,1 срещу 83,9 на сто, O3 от 4279 хеширания срещу едно, O4 от четирите порядъка разлика, O5 от 7,765 микросекунди за пълен обмен. Полезното е другаде: мерките, които се обсъждат като „скъпи“, се оказаха скъпи в брой обмена, а не в процесорно време, и това се вижда само защото решенията за секунда останаха почти непроменени.

Три неща не са измерени. Първо, плавното влошаване не се задейства при тази конфигурация, защото бисквитката вече е свалила запълването под прага; поведението му е проверено от тестовете, но не е измерено под натоварване. Второ, разпределен източник не е измерван, тъй като върху локалния интерфейс всички клиенти делят един адрес. Трето, устойчивост срещу нападател не е измервана изобщо: нито един опит с изкривен или произволно генериран вход извън построените случаи не е провеждан, тоест липсата на открит дефект тук не е доказателство за липса на дефект.

## ЗАКЛЮЧЕНИЕ

Проектът разработва система за анализ на транспортната сигурност и за управление на допускането на връзки, изградена от отделни слоеве и проверена в изолирана собствена среда. Анализаторът разчита наблюдаемата част от ръкостискането по TLS 1.3 със скорост 128 776 обмена в секунда на едно ядро. Проверката на веригите е реализирана самостоятелно по алгоритъма от [1] и дава очаквания изход за всичките тринадесет построени случая. Четирите механизма за допускане се включват поотделно, което позволи цената им да бъде измерена, а не предположена.

**Основният измерен резултат.** Собствената цена на едно решение е между 9 и 148 наносекунди, а цената на една допусната връзка под натоварване е между 454 и 1094 микросекунди. Следователно намалението на пропускателната способност при включена бисквитка, до 44,6 на сто от базата, не идва от изчислението, а от допълнителния обмен. Срещу тази цена върховото запълване на таблицата пада шест пъти, а заделената памет за състояние пет пъти, при едно и също количество лавинен трафик. Мерките, които се обсъждат като скъпи, се оказаха скъпи в брой обмени, а не в процесорно време.

**Ограничения.** Проверката на подписа не се извършва, така че системата не открива верига с правилни структурни полета и подправен подпис; това е отпечатано при всяка проверка. Пасивният анализ вижда само част от обмена, което е свойство на протокола. Опитите са на една машина и върху локалния интерфейс, където всички клиенти делят един адрес, така че ограничаването по източник не може да бъде оценено срещу разпределен източник. Плавното влошаване е проверено от тестовете, но не е измерено под натоварване. Машината е обща, поради което един ред в (Табл. 4) е отбелязан като недостатъчно устойчив. И най-същественото: системата не е подлагана на атака, на фъзинг или на външен одит, тоест нищо тук не бива да се чете като доказана устойчивост.

**По-нататъшна работа.** Проверка на подписа зад условна зависимост, така че да се включва при налична криптографска библиотека и да се отчита като непроведена, когато я няма. Измерване в среда с различни източници, което е единственият начин ограничаването по източник и плавното влошаване да бъдат оценени при условията, за които са предназначени. Фъзинг на разчитачите на TLS и на DER, тъй като те приемат вход от чужда страна. Разширяване към QUIC, където същата ключова обмяна се пренася в друг транспорт [11].

## ЛИТЕРАТУРА

- [1] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk, “Internet x.509 public key infrastructure certificate and certificate revocation list (crl) profile,” RFC 5280, Internet Engineering Task Force, May 2008.
- [2] International Organization for Standardization, Geneva, *ISO/IEC 27033-1:2015 Information technology – Security techniques – Network security – Part 1: Overview and concepts*, 2015.
- [3] International Telecommunication Union, Geneva, *Recommendation ITU-T X.509: Information technology – Open Systems Interconnection – The Directory: Public-key and attribute certificate frameworks*, October 2019.
- [4] K. McKay and D. Cooper, “Guidelines for the selection, configuration, and use of transport layer security (tls) implementations,” NIST Special Publication 800-52 Rev. 2, National Institute of Standards and Technology, August 2019.
- [5] K. Scarfone and P. Hoffman, “Guidelines on firewalls and firewall policy,” NIST Special Publication 800-41 Rev. 1, National Institute of Standards and Technology, September 2009.
- [6] European Union Agency for Cybersecurity (ENISA), “Enisa threat landscape 2023,” tech. rep., European Union Agency for Cybersecurity, October 2023.
- [7] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, “Dns security introduction and requirements,” RFC 4033, Internet Engineering Task Force, March 2005.
- [8] OWASP Foundation, *OWASP Top 10:2021 – The Ten Most Critical Web Application Security Risks*, 2021.
- [9] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” RFC 8446, Internet Engineering Task Force, August 2018.
- [10] W. Eddy, “Tcp syn flooding attacks and common mitigations,” RFC 4987, Internet Engineering Task Force, August 2007.
- [11] M. Thomson and S. Turner, “Using tls to secure quic,” RFC 9001, Internet Engineering Task Force, May 2021.
- [12] S. Santesson, M. Myers, R. Ankney, A. Malpani, S. Galperin, and C. Adams, “X.509 internet public key infrastructure online certificate status protocol – ocsp,” RFC 6960, Internet Engineering Task Force, June 2013.
- [13] P. Saint-Andre and J. Hodges, “Representation and verification of domain-based application service identity within internet public key infrastructure using x.509 (pkix) certificates in the context of transport layer security (tls),” RFC 6125, Internet Engineering Task Force, March 2011.

## ПРИЛОЖЕНИЕ

Изходният текст не е преписан тук. Той е в хранилището <https://github.com/Alex-Tsvetanov/sentinel>, заедно с пълния необобщен изход на измерванията в `docs/measurements/bench_run.txt`, върху който стъпват таблиците от Раздел IV. Заглавните файлове са в `include/sentinel/`, реализацията в `src/`, тестовете в `tests/`, програмите в `apps/`. (Табл. 6) дава картата на кода.

Таблица 6. Модули на проекта, обем и съдържание

| Модул          | Редове | Съдържание                                      |
|----------------|--------|-------------------------------------------------|
| bytes, siphash | 241    | четец с проверка на границите, SipHash-2-4      |
| tls            | 768    | записи, ръкостискане, разширения, отчет         |
| der, x509      | 1013   | строг разчитач на DER, структура на сертификата |
| chain          | 495    | изграждане на път и проверките по RFC 5280      |
| admission      | 621    | четирите механизма и политиката на влошаване    |
| net, harness   | 590    | сокети и стенд за натоварването                 |
| fixtures       | 766    | генериран стенд: байтове и сертификати          |
| experiment     | 595    | измерванията                                    |
| tests/, apps/  | 1710   | 82 случая в седем набора, двете програми        |

Пълната последователност, изпълнена на машината от Раздел III:

```
git clone https://github.com/Alex-Tsvetanov/sentinel.git sentinel-network-
↪ security
cd sentinel-network-security
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build build && ctest --test-dir build --output-on-failure
./build/sentinel_bench --duration-ms 600 --repeat 11
```

Листинг 1: Сборка, проверка и измерване

Необходими са само компилатор за C++20 и CMake от версия 3.20 нагоре. Опитите не изискват повишени права: слушателят се свързва с локалния интерфейс на непривилегирован порт, избран от операционната система.